

INFORME CORRELACIÓN ENTRE METODOS DE PROGRAMACIÓN

Brian David Acevedo Gomez

Fundamento de Computación Paralela y Distribuida

Phd. José Gerardo Chacón

Ingeniería de Sistemas – Facultad de Ingenierías y Arquitectura

Universidad de Pamplona
Villa del Rosario – Norte de Santander
2/05/2026

Tipo	Características básicas	Ventajas / fortalezas	Desventajas /debilidades	Ejemplo proyectos (link)	Relación con el proyecto de investigación
Significado: OPEN MP	Modelo de programación paralela para memoria compartida; usa directivas de compilación (#pragma omp) en C, C++ y Fortran; basado en el modelo Fork-Join donde el hilo maestro genera hilos paralelos	Fácil de implementar e integrar en código existente; paralelización incremental; no requiere gestión explícita de comunicación entre hilos; reduce tiempo de desarrollo	Solo funciona en un único nodo (memoria compartida); no escala a múltiples máquinas; no aplica a plataformas de memoria distribuida; limitado en grandes clústeres	Word count paralelo con OpenMP (C++): https://github.com/openmp	Se relaciona parcialmente: podría usarse como complemento dentro de cada nodo del clúster, pero no cubre la distribución entre nodos que exige el modelo MapReduce del proyecto
Significado: MPI	Estándar para comunicación entre procesos en sistemas de memoria distribuida ; define funciones de paso de mensajes (punto a punto y colectivas); modelo SPMD/MIMD; cada proceso tiene su propio espacio de	Portabilidad (funciona en multiprocesadores, multicomputadores, redes heterogéneas); alto rendimiento; control fino sobre la comunicación; escalabilidad horizontal real; implementaciones libres (MPICH, OpenMPI)	Programación más compleja y de bajo nivel; gestión explícita de mensajes; el acceso remoto a memoria puede ser lento; mayor curva de aprendizaje	MapReduce-MPI Library (Sandia National Lab): https://github.com/sjplimp/mapreduce	Se relaciona directamente : MPI reproduce fielmente la arquitectura distribuida de MapReduce, permitiendo implementar las fases <i>map</i> , <i>shuffle</i> y <i>reduce</i> entre nodos independientes con escalabilidad y tolerancia a fallos

	memoria e identificador				
Significado: CUDA	Plataforma de programación paralela de NVIDIA para GPUs; ejecuta miles de hilos simultáneos en núcleos GPU; soporta C, C++, Python y Fortran; organiza la ejecución en bloques e hilos (<i>kernels</i>)	Rendimiento masivamente paralelo en operaciones numéricas; lecturas dispersas de memoria; memoria compartida rápida entre hilos del mismo bloque; gran ecosistema y documentación; ideal para IA y cálculo científico	Requiere hardware NVIDIA específico; no está diseñado para procesamiento de texto heterogéneo ni para sistemas distribuidos multi-nodo; alta complejidad de programación; dependencia de hardware propietario	CUDA-MapReduce para procesamiento de imágenes: https://developer.nvidia.com/cuda-example	Se relaciona muy poco: CUDA está orientado a operaciones numéricas masivas en GPU, no al procesamiento distribuido de archivos de texto en clústeres, que es el núcleo del proyecto

De los tres modelos de programación paralela disponibles, **MPI (Message Passing Interface)** es el que mejor se ajusta al proyecto, debido a su naturaleza distribuida, su capacidad de escalar a múltiples nodos y su alineación directa con los principios del paradigma MapReduce.

Justificación del Método

MPI es el modelo que mejor responde a los objetivos del sistema MapReduce propuesto, ambos comparten la misma filosofía de procesamiento distribuido en memoria distribuida. Mientras que OpenMP opera exclusivamente sobre sistemas de memoria compartida (un solo nodo con múltiples hilos), MPI permite la ejecución en múltiples procesos distribuidos a través de diferentes nodos, lo que refleja con exactitud la arquitectura de un clúster MapReduce. En el paradigma MapReduce, los datos se fragmentan y distribuyen entre nodos independientes que no comparten memoria; MPI reproduce esta lógica mediante el paso de mensajes entre procesos, permitiendo que cada proceso actúe como un *mapper* o *reducer* autónomo. Adicionalmente, MPI ofrece control explícito sobre la comunicación entre procesos, lo que facilita implementar las fases de *shuffle* y *sort* que son propias del modelo MapReduce, y que OpenMP no puede gestionar a nivel inter-nodo. Esto garantiza escalabilidad horizontal y tolerancia a fallos básicos, dos pilares del objetivo general del proyecto.

Citas de referencia:

- Plimpton, S. J. & Devine, K. D. (2011). *MapReduce in MPI for Large-scale Graph Algorithms*. Parallel Computing, 37(9). <https://dl.acm.org/doi/abs/10.1016/j.parco.2011.02.004>[acm](#)
- Hoefer, T., Lumsdaine, A., & Dongarra, J. (2009, September). Towards efficient mapreduce using mpi. In *European Parallel Virtual Machine/Message Passing Interface Users' Group Meeting* (pp. 240-249). Berlin, Heidelberg: Springer Berlin Heidelberg.
- He, H. et al. (2015). *Performance Comparison of OpenMP, MPI, and MapReduce*. Wiley Online Library. <https://onlinelibrary.wiley.com/doi/10.1155/2015/575687>

Secuencia Lógica de Implementación con MPI

1. **Definir el entorno distribuido:** Instalar y configurar MPI (OpenMPI o MPICH) en los nodos del clúster o en una máquina local con múltiples procesos virtuales.
2. **Particionar los archivos de texto:** Dividir los archivos de entrada en bloques o fragmentos de tamaño uniforme, asignando un fragmento a cada proceso MPI (equivalente a la fase de *splitting*).
3. **Implementar la función map():** Cada proceso MPI recibe su fragmento, lo analiza (ej. conteo de palabras, frecuencia de tokens) y genera pares clave-valor intermedios de forma independiente.
4. **Implementar la fase de Shuffle & Sort*:** Usar operaciones de comunicación colectiva de MPI (MPI_Gather, MPI_Alltoall) para redistribuir los pares clave-valor, agrupando todas las entradas con la misma clave en el mismo proceso reductor.<http://tor.inf.ethz>

5. **Implementar la función `reduce()`:** Cada proceso reductor recibe las listas de valores asociadas a sus claves asignadas y aplica la operación de agregación (suma, conteo, promedio, etc.).
6. **Consolidar los resultados:** El proceso raíz (rank 0) recopila las salidas de todos los reductores mediante `MPI_Gather` o escritura directa a disco, generando el resultado final.
7. **Implementar tolerancia a fallos básica:** Añadir manejo de errores con `MPI_Errhandler` y diseñar la lógica de reintento o reasignación de tareas cuando un proceso falle durante las fases map o reduce. [sjplimp.github](https://github.com/sjplimp)
8. **Realizar pruebas de rendimiento:** Medir el *speedup* y la eficiencia al incrementar el número de procesos MPI y el tamaño de los archivos, comparando contra una versión secuencial como línea base. [sahiljaganmohan+1](#)

Conclusiones

Tras revisar los tres modelos de programación paralela, se puede afirmar que MPI es la opción que mejor encaja con lo que este proyecto necesita. La razón principal es simple: MapReduce nació para funcionar en entornos donde los datos se distribuyen entre múltiples máquinas que no comparten memoria, y MPI trabaja exactamente bajo esa misma lógica.

OpenMP es una herramienta útil y más fácil de aprender, pero su alcance se limita a un solo equipo, lo que lo hace insuficiente cuando el objetivo es procesar archivos de texto de forma distribuida entre varios nodos. CUDA, aunque poderoso, está pensado para un tipo de problema muy diferente: cálculos numéricos masivos en tarjetas gráficas, no para analizar texto en un clúster.

MPI permite que cada proceso actúe de manera independiente, se comunique con los demás mediante paso de mensajes y escale según crezca la carga de trabajo, lo cual refleja fielmente cómo opera un sistema MapReduce real. Por eso, para un proyecto que busca demostrar de forma práctica los principios de paralelismo, distribución y escalabilidad, MPI no es simplemente una opción viable, sino la más coherente con los fundamentos teóricos y los objetivos planteados.